

시스템 프로그래밍

리눅스&유닉스

[강의교안 이용 안내]

- 본 강의교안의 저작권은 한빛아카데미(주)에 있습니다.
- 이 자료를 무단으로 전제하거나 배포할 경우 저작권법 136조에 의거하여 처벌을 받을 수 있습니다.

시스템 프로그래밍

리눅스&유닉스

Chapter 04 파일 입출력

목차

01 개요

02 저수준 파일 입출력

03 고수준 파일 입출력

04 파일 기술자와 파일 포인터 변환

05 임시 파일 사용

학습목표

- 파일 입출력의 특징을 이해한다.
- 저수준의 파일 입출력 함수를 사용할 수 있다.
- 고수준의 파일 입출력 함수를 사용할 수 있다.
- 임시 파일을 생성해 파일 입출력을 할 수 있다.

01. 개요

■ 파일

■ 파일

- 관련 있는 데이터의 집합으로, 저장 장치에 일정한 형태로 저장
- 데이터를 저장하는 데는 물론 데이터를 전송하거나 장치에 접근하는 데도 사용
- 리눅스에서 파일은 크게 일반 파일과 특수 파일로 구분
- 특수 파일의 생성과 삭제 및 입출력은 특수 파일별로 약간씩 차이가 있음

표 4-1 파일의 종류

종류	용도
일반 파일	텍스트나 바이너리 형태의 자료를 저장하는 파일
특수 파일	데이터 전송 또는 장치 접근에 사용하는 파일

01. 개요

■ 파일

■ 파일 읽고 쓰는 방법

• 저수준 파일 입출력

- 리눅스 커널의 시스템 호출을 이용해 파일 입출력을 수행
- 시스템 호출을 이용하므로 파일에 좀 더 빠르게 접근할 수 있는 장점
- 또한 바이트 단위로 파일의 내용을 다루므로 일반 파일뿐만 아니라 특수 파일도 읽고 쓸 수 있음
- 바이트 단위로만 입출력을 수행 가능 하므로 응용프로그램 작성시 다른 추가기능을 함수로 추가 구현 해야함
- 열린 파일을 참조할 때 파일 기술자 사용

• 고수준 파일 입출력

- 저수준 파일 입출력의 불편함을 해결하기 위해 제공
- C 언어의 표준 함수로 제공
- 데이터를 바이트 단위로 한정하지 않고 버퍼를 이용해 한꺼번에 읽기와 쓰기를 수행
- 다양한 입출력 데이터 변환 기능도 이미 구현되어 있어 자료형에 따라 편리하게 이용할 수 있음
- 열린 파일을 참조할 때 파일 포인터 사용

01. 개요

■ 파일

■ 파일 읽고 쓰는 방법

표 4-2 저수준 파일 입출력과 고수준 파일 입출력 비교

	저수준 파일 입출력	고수준 파일 입출력
파일 지시자	int fd	FILE *fp;
특징	- 훨씬 빠름 - 바이트 단위로 읽고 쓰기 - 특수 파일에 대한 접근 가능	- 사용하기 쉬움 - 버퍼 단위로 읽고 쓰기 - 데이터의 입출력 동기화가 쉬움 - 여러 가지 형식을 지원
주요 함수	open(), close(), read(), write(), dup(), dup2(), fcntl(), lseek(), fsync()	fopen(), fclose(), fread(), fwrite(), fputs(), fgets(), fprintf(), fscanf(), freopen(), fseek()

02. 저수준 파일 입출력

■ 파일 기술자

■ 파일 기술자

- 현재 열려 있는 파일을 구분할 목적으로 시스템에서 붙여놓은 번호
- 저수준 파일 입출력에서는 열 린 파일을 참조하는 데 사용하는 지시자 역할 수행
- 파일 기술자는 정숫값으로, `open()` 함수를 사용해 파일을 열 때 부여
- 프로세스가 파일을 열 때 파일 기술자에는 0번부터 순서대로 가장 작은 번호가 자동 할당
- 0번부터 시작하여 0번, 1번, 2번 파일 기술자에는 기본적인 용도가 지정되어 있음
 - 0번: 표준 입력
 - 1번: 표준 출력
 - 2번: 표준 오류 출력

파일 기술자

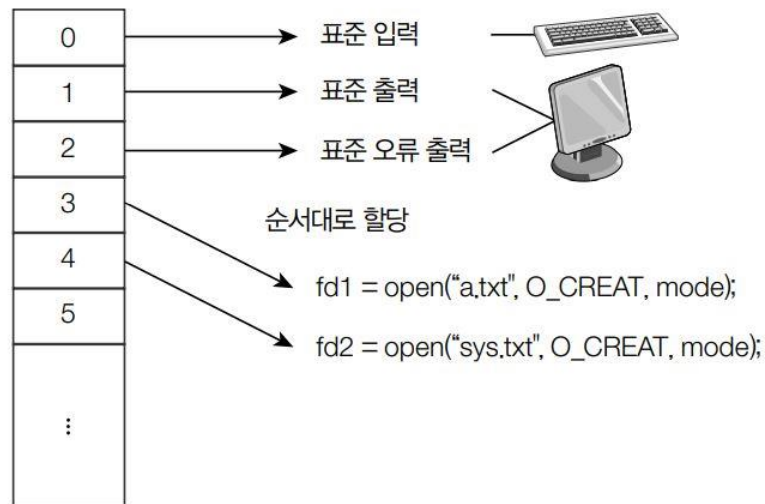


그림 4-1 파일 기술자 할당

02. 저수준 파일 입출력

■ 파일 열기 : open(2)

```
#include <sys/types.h>
```

[함수 원형]

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int open(const char *pathname, int flags);
```

```
int open(const char *pathname, int flags, mode_t mode);
```

- pathname : 열려는 파일이 있는 경로
- flags : 파일 상태 플래그
- mode : 접근 권한
- open() 함수의 특징
 - pathname에 지정한 파일을 flags에 지정한 상태 플래그의 값에 따라 열고 파일 기술자를 반환
 - 파일의 상태를 조정 하는 flags 값은 man 명령으로 확인할 수 있음

02. 저수준 파일 입출력

■ 파일 열기 : open(2)

- flags의 주요 플래그

표 4-3 flags에 사용하는 주요 플래그

종류	기능
O_RDONLY	파일을 읽기 전용으로 연다.
O_WRONLY	파일을 쓰기 전용으로 연다.
O_RDWR	파일을 읽기/쓰기용으로 연다.
O_CREAT	파일이 없으면 생성한다. 파일을 생성할 권한은 당연히 있어야 한다. 파일이 이미 있다면 아무 의미 없는 옵션이다.
O_EXCL	O_CREAT 옵션과 함께 사용할 경우 기존에 없는 파일이면 파일을 생성하고, 이미 있으면 파일을 생성하지 않고 오류 메시지를 출력한다.
O_APPEND	이 옵션을 지정하면 파일의 맨 끝에 내용을 추가한다.
O_TRUNC	파일을 생성할 때 이미 있는 파일이고 쓰기 옵션으로 열었으면 내용을 모두 지우고 파일 길이를 0으로 변경한다.
O_SYNC/ O_DSYNC	파일에 쓰기 동작을 할 때는 보통 버퍼에만 쓰고 나중에 디스크와 같은 저장 장치로 옮겨 쓰는데, 이 옵션이 설정되어 있으면 저장 장치에 쓰기를 마쳐야 쓰기 동작을 완료한다. O_SYNC 플래그는 파일의 수정 시각 속성도 수정할 때까지 기다린다. 이 옵션을 설정 하면 프로그램 실행 속도는 느려질 수 있지만 디스크에 확실하게 저장됨을 보장한다.

02. 저수준 파일 입출력

■ 파일 열기 : open(2)

- 플래그 지정

- 쓰기 전용으로 열 때(이미 파일이 있는 경우)

```
O_WRONLY | O_TRUNC
```

- 쓰기 전용으로 열 때(파일이 없는 경우)

```
O_WRONLY | O_CREAT | O_TRUNC
```

- 읽기/쓰기/추가용으로 열 때

```
O_RDWR | O_APPEND
```

- 상수를 이용해 0644 권한을 지정

```
mode = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
```

02. 저수준 파일 입출력

■ 파일 생성 : creat(2)

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

[함수 원형]

```
int creat(const char *pathname, mode_t mode);
```

- pathname : 파일을 생성할 경로
- mode : 접근 권한
- creat() 함수의 특징
 - open() 함수에 파일 생성 기능이 없던 구 버전 유닉스에서 사용하던 것
 - open() 함수와 달리 플래그를 지정하는 부분이 없음
 - creat() 함수로 파일을 생성하면 파일 기술자가 리턴되므로 별도로 open() 함수를 호출해 파일을 열 필요가 없음
- 다음에서 creat() 함수와 open() 함수는 같은 의미

```
creat(pathname, mode);
open(pathname, O_CREAT | O_WRONLY | O_TRUNC, mode);
```

02. 저수준 파일 입출력

■ 파일 닫기 : close(2)

```
#include <unistd.h>
```

[함수 원형]

```
int close(int fd);
```

- fd : 파일 기술자
- close() 함수의 특징
 - 한 프로세스가 열 수 있는 파일 개수에 제한이 있으므로 파일 입출력 작업을 모두 완료하면 반드시 파일을 닫아야 함
 - 파일을 성공적으로 닫으면 0을 리턴
 - 파일 닫기에 실패하면 -1을 리턴하고 오류 코드를 외부 변수 errno에 저장

02. 저수준 파일 입출력

■ [예제 4-1] 새 파일 생성하기

```
01 #include <sys/types.h>
02 #include <sys/stat.h>
03 #include <fcntl.h>
04 #include <unistd.h>
05 #include <stdlib.h>
06 #include <stdio.h>
07
08 int main() {
09     int fd;
10     mode_t mode;
11
12     mode = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
13
14     fd = open("test.txt", O_CREAT, mode);
15     if (fd == -1) {
16         perror("Creat");
17         exit(1);
18     }
19     close(fd);
20 }
```

실행

```
$ ls test.txt
ls: 'test.txt'에 접근할 수 없습니다: 그런 파일이나 디렉터리가 없습니다
$ ch4_1.out
$ ls -l test.txt
-rw-r--r-- 1 jw jw 0  3월  6 22:57 test.txt
```

- **12행** 파일 접근 권한을 OR 연산으로 지정
- **14행** test.txt 파일을 12행에서 지정한 권한(0644)으로 생성
- **15~18행** 파일 생성 도중 오류가 발생하면 리턴값이 -1이 되므로 16행에서 perror() 함수를 사용해 오류 메시지를 출력
17행에서는 exit() 함수를 사용해 프로그램의 실행을 종료
- **실행 결과** test.txt 파일이 생성되었고 접근 권한이 0644임을 알 수 있음

02. 저수준 파일 입출력

■ [예제 4-2] O_EXCL 플래그 사용하기

```
01 #include <sys/types.h>
02 #include <sys/stat.h>
03 #include <fcntl.h>
04 #include <unistd.h>
05 #include <stdlib.h>
06 #include <stdio.h>
07
08 int main() {
09     int fd;
10
11     fd = open("test.txt", O_CREAT | O_EXCL, 0644);
12     if (fd == -1) {
13         perror("Excl");
14         exit(1);
15     }
16     close(fd);
17 }
```

실행

```
$ ls test.txt
test.txt
$ ch4_2.out
Excl: File exists
```

```
$ rm test.txt
$ ch4_2.out
$ ls test.txt
test.txt
```

- **11행** 모드를 0644와 같이 숫자로 지정하고 O_EXCL 플래그를 같이 지정
이 경우 test.txt 파일이 이미 있으면 오류 메시지를 출력
- **12~15행** 파일 생성 도중 오류가 발생하면 오류 메시지를 출력하고 프로그램의 실행을 종료
- **첫 번째 실행 결과** 파일이 있을 경우 "Excl: File exists"라는 오류 메시지를 출력
test.txt파일은 [예제 4-1]에서 이미 생성했으므로 오류가 발생한 것
- **두 번째 실행 결과** 파일을 지운 후 다시 실행하면 파일을 생성

02. 저수준 파일 입출력

■ [예제 4-2] O_EXCL 플래그 사용하기

```
01 #include <sys/types.h>
02 #include <sys/stat.h>
03 #include <fcntl.h>
04 #include <unistd.h>
05 #include <stdlib.h>
06 #include <stdio.h>
07
08 int main() {
09     int fd;
10
11     fd = open("test.txt", O_CREAT | O_EXCL, 0644);
12     if (fd == -1) {
13         perror("Excl");
14         exit(1);
15     }
16     close(fd);
17 }
```

- 16행에서 파일을 닫기 전에 다음과 같은 printf 문을 추가해 파일 기술자를 출력

```
printf("test.txt : fd = %d\n", fd);
```

실행

```
$ rm test.txt
$ ch4_2.out
test.txt: fd = 3
```


02. 저수준 파일 입출력

■ [예제 4-3] 파일 기술자 할당하기

```
01 #include <sys/types.h>
02 #include <sys/stat.h>
03 #include <fcntl.h>
04 #include <unistd.h>
05 #include <stdlib.h>
06 #include <stdio.h>
07
08 int main() {
09     int fd;
10
11     close(0);
12
13     fd = open("test.txt", O_RDWR);
14     if (fd == -1) {
15         perror("Open");
16         exit(1);
17     }
18
19     printf("test.txt : fd = %d\n", fd);
20     close(fd);
21 }
```

실행

```
$ ch4_3.out
test.txt : fd = 0
```

- **11행** 0번 파일 기술자를 닫음
- **13행** 남아 있는 파일 기술자 중 가장 작은 숫자가 0이므로 test.txt 파일을 열고, 할당되는 파일 기술자는 실행 결과와 같이 0이 됨

02. 저수준 파일 입출력

■ 파일 읽기 : read(2)

```
#include <unistd.h>
```

[함수 원형]

```
ssize_t read(int fd, void *buf, size_t count);
```

- fd : 파일 기술자
- buf : 파일에 기록할 데이터를 저장한 메모리 영역
- count : buf의 크기(기록할 데이터의 크기)
- read() 함수의 특징
 - 파일 기술자가 가리키는 파일에서 count에 지정한 크기만큼 바이트를 읽어 buf로 지정한 메모리 영역에 저장
 - 실제로 읽은 바이트 수를 리턴하며, 오류가 발생하면 -1을 리턴
 - 만일 리턴값이 0이면 파일의 끝에 도달해 더 이상 읽을 내용이 없음을 의미
 - 파일에 저장된 데이터가 텍스트든 이미지든 상관없이 무조건 바이트 단위로 읽음

02. 저수준 파일 입출력

■ [예제 4-4] 파일 읽기

```
01 #include <fcntl.h>
02 #include <unistd.h>
03 #include <stdlib.h>
04 #include <stdio.h>
05
06 int main() {
07     int fd, n;
08     char buf[10];
09
10     fd = open("linux.txt", O_RDONLY);
11     if (fd == -1) {
12         perror("Open");
13         exit(1);
14     }
15
16     n = read(fd, buf, 5);
17     if (n == -1) {
18         perror("Read");
19         exit(1);
20     }
21
22     buf[n] = '\0';
23     printf("n=%d, buf=%s\n", n, buf);
24     close(fd);
25 }
```

실행

```
$ cat linux.txt
Linux System Programming
$ ch4_4.out
n=5, buf=Linux
```

- **10행** linux.txt 파일을 읽기 전용으로 실행
- **16행** 5바이트를 읽어 buf에 저장하고 read() 함수의 리턴값을 저장
출력 결과를 보면 5임을 알 수 있음
- **22행** read() 함수는 읽어온 데이터의 끝에 자동으로 널을 추가하지 않기 때문에 buf에 널 문자('\0')를 추가
buf를 문자열로 출력하려면 널 문자를 추가해야 함
- **23행** linux.txt 파일에서 읽어온 바이트 수와 buf의 내용을 출력
실행 결과를 보면 linux.txt 파일의 처음 5바이트를 읽어왔음을 알 수 있음
- 처음 파일의 오프셋은 'L'을 나타내지만 (오프셋 0)
read() 함수를 실행한 다음에는 5바이트 이동해 S 앞의 공백 문자(' ')를 가리킴 (오프셋 5)
따라서 계속 읽어오면 ' '부터 읽음

02. 저수준 파일 입출력

■ 파일 쓰기 : write(2)

```
#include <unistd.h>
```

[함수 원형]

```
ssize_t write(int fd, const void *buf, size_t count);
```

- fd : 파일 기술자
- buf : 파일에 기록할 데이터를 저장한 메모리 영역
- count : buf의 크기(기록할 데이터의 크기)
- write() 함수의 특징
 - read() 함수와 인자의 구조는 같지만 의미가 다름
 - 파일 기술자는 쓰기를 수행할 파일을 가리키고, buf는 파일에 기록할 데이터를 저장하고 있는 메모리 영역을 가리킴
 - buf가 가리키는 메모리 영역에서 count로 지정한 크기만큼 읽어 파일에 쓰기를 수행
 - write() 함수도 실제로 쓰기를 수행한 바이트 수를 리턴하며, 오류가 발생하면 -1을 리턴

02. 저수준 파일 입출력

■ [예제 4-5] 파일 쓰기

```
01 #include <fcntl.h>
02 #include <unistd.h>
03 #include <stdlib.h>
04 #include <stdio.h>
05
06 int main() {
07     int rfd, wfd, n;
08     char buf[10];
09
10     rfd = open("linux.txt", O_RDONLY);
11     if(rfd == -1) {
12         perror("Open linux.txt");
13         exit(1);
14     }
15
16     wfd = open("linux.bak", O_CREAT | O_WRONLY | O_TRUNC, 0644);
17     if (wfd == -1) {
18         perror("Open linux.bak");
19         exit(1);
20     }
21
22     while ((n = read(rfd, buf, 6)) > 0) {
23         if (write(wfd, buf, n) != n) perror("Write");
24     }
25
26     if (n == -1) perror("Read");
27
28     close(rfd);
29     close(wfd);
30 }
```

실행

```
$ ls linux.bak
```

```
ls: 'linux.bak'에 접근할 수 없습니다: 그런 파일이나 디렉터리가 없습니다
```

```
$ ch4_5.out
```

```
$ cat linux.bak
```

```
Linux System Programming
```

- **10행** linux.txt 파일을 읽기 전용으로 열고, 파일 기술자를 rfd에 저장
- **16행** linux.bak 파일을 쓰기 전용으로 생성하고, 기존 파일이 있으면 내용을 비움
- **22~24행** while 문을 사용해 linux.txt 파일을 6바이트씩 읽고 출력
22행에서 read() 함수가 리턴한 값이 buf에 저장된 데이터의 크기이므로
이를 23행에서 write() 함수의 세 번째 인자로 사용할 수 있다.
write() 함수의 리턴값이 출력할 데이터의 크기인 n과 다르면 쓰기 동작에
문제가 있다는 의미이므로 오류 메시지를 출력
- **실행 결과** linux.bak 파일이 생성되고 데이터가 저장
즉 linux.txt 파일이 linux.bak 파일로 복사

02. 저수준 파일 입출력

■ 파일 오프셋 위치 지정 : lseek(2)

```
#include <sys/types.h>
```

[함수 원형]

```
#include <unistd.h>
```

```
off_t lseek(int fd, off_t offset, int whence);
```

- fd : 파일 기술자
- offset : 이동할 오프셋 위치
- whence : 오프셋의 기준 위치

• lseek() 함수의 특징

- lseek() 함수는 파일 기술자가 가리키는 파일에서 offset으로 지정한 크기만큼 오프셋을 이동
- 이때 offset의 값은 whence의 값을 기준으로 해석

표 4-4 오프셋의 기준 위치를 나타내는 whence 값

값	설명
SEEK_SET	파일의 시작을 기준으로 계산
SEEK_CUR	현재 위치를 기준으로 계산
SEEK_END	파일의 끝을 기준으로 계산

02. 저수준 파일 입출력

■ 파일 오프셋 위치 지정 : lseek(2)

- lseek() 함수의 특징

- 파일의 시작에서 다섯 번째 위치로 이동

```
lseek(fd, 5, SEEK_SET);
```

- 파일의 끝에서 0번째이므로 파일 끝으로 이동

```
lseek(fd, 0, SEEK_END);
```

- 현재 위치를 기준으로 0만큼 이동한 값을 구해 파일 오프셋 위치를 확인

```
cur_offset = lseek(fd, 0, SEEK_CUR);
```

02. 저수준 파일 입출력

■ [예제 4-6] 파일 오프셋 사용하기

```
01 #include <sys/types.h>
02 #include <fcntl.h>
03 #include <unistd.h>
04 #include <stdlib.h>
05 #include <stdio.h>
06
07 int main() {
08     int fd, n;
09     off_t start, cur;
10     char buf[256];
11
12     fd = open("linux.txt", O_RDONLY);
13     if (fd == -1) {
14         perror("Open linux.txt");
15         exit(1);
16     }
17
18     start = lseek(fd, 0, SEEK_CUR);
19     n = read(fd, buf, 255);
20     buf[n] = '\0';
21     printf("Offset start=%d, n=%d, Read Str=%s", (int)start, n, buf);
22     cur = lseek(fd, 0, SEEK_CUR);
23     printf("Offset cur=%d\n", (int)cur);
24
25     start = lseek(fd, 6, SEEK_SET);
26     n = read(fd, buf, 255);
27     buf[n] = '\0';
28     printf("Offset start=%d, Read Str=%s", (int)start, buf);
29
30     close(fd);
31 }
```

실행

```
$ ch2_6.out
```

```
Offset start=0, n=25, Read Str=Linux System Programming
```

```
Offset cur=25
```

```
Offset start=6, Read Str=System Programming
```

- **12행** linux.txt 파일을 읽기 전용으로 실행
- **18행** 파일의 현재 오프셋 위치를 파악해보면 0임을 알 수 있음
- **19~21행** 파일을 읽고 마지막에 널 값을 넣어 출력
n의 값이 25로 출력되었는데, 이는 읽은 글자 수 + 널 문자 개수
- **22행** 파일에서 데이터를 읽은 후 현재 위치를 확인
(이 예제에서는 오프셋이 start + n 위치로 이동)
- **25~28행** 파일의 시작을 기준으로 오프셋이 6인 위치로 이동한 후 데이터를
읽고 널 값을 채워 출력
- **실행 결과** 오프셋의 위치에 따라 읽어온 데이터가 다름을 알 수 있음

02. 저수준 파일 입출력

■ 파일 기술자 복사 : dup(2)

```
#include <unistd.h>
```

[함수 원형]

```
int dup(int oldfd);
```

- oldfd : 복사할 파일 기술자
- dup() 함수의 특징
 - 기존 파일 기술자를 인자로 받아 새로운 파일 기술자를 리턴
 - 이때 새로 할당되는 파일 기술자는 현재 할당할 수 있는 파일 기술자 중 가장 작은 값으로 자동 할당
 - 파일 기술자의 복사는 입출력 방향 전환에 많이 사용

02. 저수준 파일 입출력

■ [예제 4-7] 파일 기술자 복사하기

```
01 #include <fcntl.h>
02 #include <unistd.h>
03 #include <stdlib.h>
04 #include <stdio.h>
05
06 int main() {
07     int fd, fd1;
08
09     fd = open("tmp.aaa", O_CREAT | O_WRONLY | O_TRUNC, 0644);
10     if (fd == -1) {
11         perror("Create tmp.aaa");
12         exit(1);
13     }
14
15     close(1);
16
17     fd1 = dup(fd);
18     printf("DUP FD=%d\n", fd1);
19     printf("Standard Output Redirection\n");
20
21     close(fd);
22 }
```

실행

```
$ ch4_7.out
$ cat tmp.aaa
DUP FD=1
Standard Output Redirection
```

- **9행** tmp.aaa 파일을 생성
- **15행** 파일 기술자 1번을 닫음, 1번은 표준 출력 즉, 화면으로 출력되는 것을 닫은 것
- **17행** dup(fd)를 실행하면 파일 기술자 fd가 가리키는 파일에 새로운 파일 기술자가 지정
15행에서 1번을 닫았기 때문에 현재 가장 작은 값은 1이고, 이 값이 새로운 파일 기술자로 할당
- **18~19행** printf 문으로 출력하면 현재 1번 파일 기술자가 가리키는 파일인 tmp.aaa로 출력
- **실행 결과** 18행과 19행에서 출력한 내용이 tmp.aaa 파일에 저장

02. 저수준 파일 입출력

■ 파일 기술자 복사 : dup2(3)

```
#include <unistd.h>
```

[함수 원형]

```
int dup2(int oldfd, int newfd);
```

- oldfd : 복사할 파일 기술자
- newfd : 파일 기술자를 복사할 곳
- dup2() 함수의 특징
 - dup() 함수와 달리 새로운 파일 기술자를 지정할 수 있게 해줌
 - 파일 기술자 oldfd를 newfd로 복사

02. 저수준 파일 입출력

■ [예제 4-8] dup2() 함수로 파일 기술자 복사하기

```
01 #include <fcntl.h>
02 #include <unistd.h>
03 #include <stdlib.h>
04 #include <stdio.h>
05
06 int main() {
07     int fd;
08
09     fd = open("tmp.bbb", O_CREAT | O_WRONLY | O_TRUNC, 0644);
10     if (fd == -1) {
11         perror("Create tmp.bbb");
12         exit(1);
13     }
14
15     dup2(fd, 1);
16     printf("DUP2: Standard Output Redirection\n");
17
18     close(fd);
19 }
```

실행

```
$ ch4_8.out
```

```
$ cat tmp.bbb
```

```
DUP2: Standard Output Redirection
```

- **15행** dup2() 함수를 사용해 기존 파일 기술자(fd)를 1번 파일 기술자로 복사
이제 1번 파일 기술자로 출력하면 더 이상 표준 출력(화면)이 아닌 fd가 가리키는 파일로 출력

02. 저수준 파일 입출력

■ 파일 기술자 제어: fcntl(2)

```
#include <unistd.h>
```

[함수 원형]

```
#include <fcntl.h>
```

```
int fcntl(int fd, int cmd, ... /* arg */);
```

- fd : 파일 기술자
 - cmd : 명령
 - arg : cmd에 따라 필요시 지정하는 인자들
-
- fcntl() 함수의 특징
 - 파일 기술자가 가리키는 파일에 cmd로 지정한 명령을 수행
 - cmd의 종류에 따라 인자를 지정해야 할 수도 있다.
 - fcntl() 함수의 두 번째 인자인 명령은 fcntl.h 파일에 정의되어 있으며 다음 두 가지를 자주 사용
 - **F_GETFL** : 상태 플래그 정보를 읽어옴
 - **F_SETFL** : 상태 플래그 정보를 설정, 설정할 수 있는 것은 대부분 open() 함수에서 지정하는 플래그

02. 저수준 파일 입출력

■ [예제 4-9] fcntl() 함수로 파일 기술자 제어하기

```
01 #include <sys/types.h>
02 #include <fcntl.h>
03 #include <unistd.h>
04 #include <stdlib.h>
05 #include <stdio.h>
06
07 int main() {
08     int fd, flags;
09
10     fd = open("linux.txt", O_RDWR);
11     if (fd == -1) {
12         perror("open");
13         exit(1);
14     }
15
16     if ((flags = fcntl(fd, F_GETFL)) == -1) {
17         perror("fcntl");
18         exit(1);
19     }
20
21     flags |= O_APPEND;
22
23     if (fcntl(fd, F_SETFL, flags) == -1) {
24         perror("fcntl");
25         exit(1);
26     }
27
28     if (write(fd, "Hanbit Academy\n", 15) != 15) perror("write");
29
30     close(fd);
31 }
```

실행

```
$ cat linux.txt
Linux System Programming
$ ch4_9.out
$ cat linux.txt
Linux System Programming
Hanbit Academy
```

- **10행** linux.txt 파일을 읽기/쓰기용으로 실행
- **16행** F_GETFL 명령을 설정해 fcntl() 함수를 호출하면 현재 설정된 플래그 값이 리턴
- **21행** 리턴된 플래그 값에 변경할 플래그를 OR로 연결해 새로운 플래그로 저장
- **23행** 저장된 플래그를 F_SETFL 명령으로 설정해 fcntl() 함수를 호출하면 기존 플래그가 새로운 플래그로 변경
- **28행** 파일 설정을 O_APPEND로 변경했기 때문에 파일에 출력하면 내용이 파일 끝부분에 추가. 이는 실행 결과에서 확인할 수 있음

02. 저수준 파일 입출력

■ 파일 삭제: remove(3)

```
##include <stdio.h>
```

[함수 원형]

```
int remove(const char *pathname);
```

- remove() 함수의 특징
 - path에 지정한 파일이나 디렉터리를 삭제
 - 내부적으로 삭제 대상이 파일이면 unlink() 함수를 호출
 - 삭제 대상이 디렉터리면 rmdir() 함수를 호출
 - 디렉터리는 비어 있을 때만 삭제

02. 저수준 파일 입출력

■ [예제 4-10] remove() 함수로 파일 삭제하기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     int ret;
06
07     ret = remove("tmp.bbb");
08     if (ret == -1) {
09         perror("Remove tmp.bbb");
10         exit(1);
11     }
12
13     printf("Remove tmp.bbb success!!!\n");
14 }
```

실행

```
$ ls tmp.*
tmp.aaa tmp.bbb
$ ch4_10.out
Remove tmp.bbb success!!!
$ ls tmp.*
tmp.aaa
```

- **07행** remove() 함수를 이용해 tmp.bbb 파일을 삭제
- **13행** 파일을 성공적으로 삭제하면 "Remove tmp.bbb success!!!" 메시지를 출력
- **실행 결과** tmp.bbb 파일이 삭제

02. 저수준 파일 입출력

■ 파일과 디스크 동기화 함수 fsync(3)

```
#include <unistd.h>
```

[함수 원형]

```
int fsync(int fd);
```

- fd : 디스크로 저장할 파일의 파일 기술자
- fsync() 함수의 특징
 - 메모리에 위치하고 있는 파일의 내용을 디스크로 보내 메모리와 디스크의 내용을 동기화
 - 메모리의 내용이 디스크로 모두 기록되기 전에는 리턴하지 않음

03. 고수준 파일 입출력

■ 파일 포인터

■ 파일 포인터

- 열린 파일을 가리킬 때 고수준 파일 입출력에서는 파일 포인터를 사용
- 파일 포인터는 디스크에서 메모리로 읽어온 파일의 위치(주소)에 관한 정보를 담고 있는 포인터
- 파일 기술자는 정수형이지만 파일 포인터는 시스템 헤더 파일에 정의되어 있는 FILE 포인터
- 플랫폼 독립적인 구조이므로 어느 플랫폼에서든 동일한 동작을 수행한다는 장점

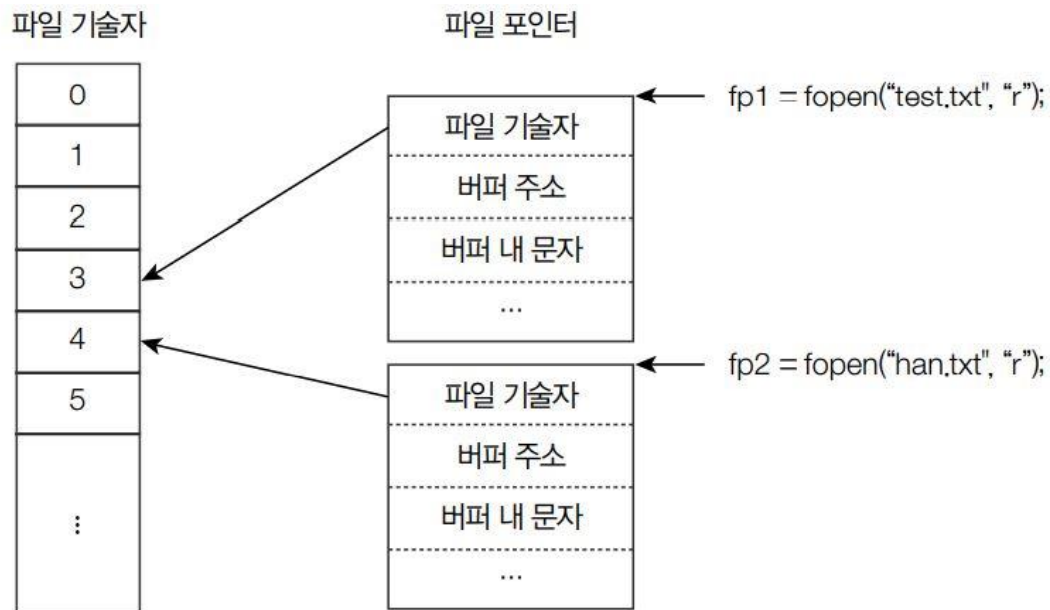


그림 4-2 파일 포인터와 파일 기술자

03. 고수준 파일 입출력

■ 파일 열기 : fopen(3)

```
#include <stdio.h>
```

[함수 원형]

```
FILE *fopen(const char *pathname, const char *mode)
```

- pathname : 파일의 경로
- mode : 파일 열기 모드
- fopen() 함수의 특징
 - 고수준 파일 입출력에서 파일을 여는 함수
 - pathname으로 지정한 파일을 mode에서 지정한 모드로 열고 파일 포인터를 리턴
 - fopen() 함수가 성공적으로 수행되면 열린 파일의 주소를 파일 포인터로 리턴하고, 실패하면 0을 리턴

```
FILE *fp;  
fp = fopen("test.txt", "r");
```

- 이 경우 test.txt 파일을 읽기 전용으로 열고 파일 포인터 주소를 파일 포인터 변수인 fp에 저장

03. 고수준 파일 입출력

■ 파일 열기 : fopen(3)

- fopen() 함수의 특징
 - 두 번째 인자인 mode에 사용할 수 있는 값

표 4-5 fopen() 함수의 열기 모드

모드	의미
r	읽기 전용으로 텍스트 파일을 연다.
w	새로 쓰기용으로 텍스트 파일을 연다. 기존 내용은 삭제된다.
a	기존 내용의 끝에 추가해서 쓰기용으로 텍스트 파일을 연다.
rb	읽기 전용으로 바이너리 파일을 연다.
wb	새로 쓰기용으로 바이너리 파일을 연다. 기존 내용은 삭제된다.
ab	추가해서 쓰기용으로 바이너리 파일을 연다.
r+	읽기와 쓰기용으로 텍스트 파일을 연다.
w+	쓰기와 읽기용으로 텍스트 파일을 연다.
a+	추가 쓰기와 읽기용으로 텍스트 파일을 연다.
rb+	읽기와 쓰기용으로 바이너리 파일을 연다.
wb+	쓰기와 읽기용으로 바이너리 파일을 연다.
ab+	추가해서 쓰기와 읽기용으로 바이너리 파일을 연다.

03. 고수준 파일 입출력

■ 파일 닫기 : fclose(3)

```
#include <stdio.h>
```

[함수 원형]

```
int fclose(FILE *stream);
```

- stream : fopen()에서 리턴한 파일 포인터
- fclose() 함수의 특징
 - fopen() 함수에서 리턴한 파일 포인터를 인자로 지정한 후 메모리에 있는 파일 내용을 디스크에 저장하고 종료
 - 파일 닫기를 정상적으로 완료하면 0을, 도중에 오류가 발생하면 EOF(-1)를 리턴

```
FILE *fp;  
fp = fopen("test.txt", "r");  
fclose(fp);
```

03. 고수준 파일 입출력

■ 문자 기반 입력 함수 : fgetc(3), getc(), getchar(), getw(3)

```
#include <stdio.h>
```

[함수 원형]

```
int fgetc(FILE *stream);
```

```
int getc(FILE *stream);
```

```
int getchar(void);
```

```
int getw(FILE *stream);
```

- stream : 파일 포인터
- fgetc(), getc(), getchar(), getw() 함수의 특징
 - fgetc() 함수: 파일 포인터가 가리키는 파일로부터 문자 한 개를 unsigned char 형태로 읽어옴
 - getc() 함수: 매크로로 구현되어 있어 실행 속도는 약간 빠르지만 실행 코드가 확장되므로 메모리를 조금 더 차지
그 외에는 fgetc() 함수와 동일한 기능을 수행
 - getchar() 함수: 표준 입력에서 문자 한 개를 읽어오는 매크로로, getc(stdin)과 같음
 - getw() 함수는 파일에서 워드 단위로 읽음, 워드의 크기는 int형의 크기로 시스템에 따라 달라짐

03. 고수준 파일 입출력

■ 문자 기반 출력 함수 : fputc(3), putc(), putchar(), putw(3)

```
#include <stdio.h>
```

[함수 원형]

```
int fputc(int c, FILE *stream);  
int putc(int c, FILE *stream);  
int putchar(int c);  
int putw(int w, FILE *stream);
```

- c, w : 출력할 문자
- stream : 파일 포인터
- fputc(), putc(), putchar(), putw() 함수의 특징
 - fputc() 함수: 인자로 받은 int형 데이터 c를 unsigned char로 변환해 파일에 사용
 - putc() 함수: fputc() 함수와 같은 동작을 수행하며 getc()와 마찬가지로 매크로로 구현 되어있음
 - putchar() 함수: 표준 출력으로 한 문자를 출력하는 매크로, putc(c, stdout)와 같음
 - putw() 함수: 워드 단위로 파일에 출력, 워드의 크기는 int형의 크기로 시스템에 따라 달라짐

03. 고수준 파일 입출력

■ [예제 4-11] 문자 기반 입출력 함수 사용하기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     FILE *rfp, *wfp;
06     int c;
07
08     if ((rfp = fopen("linux.txt", "r")) == NULL) {
09         perror("fopen: linux.txt");
10         exit(1);
11     }
12
13     if ((wfp = fopen("linux.out", "w")) == NULL) {
14         perror("fopen: linux.out");
15         exit(1);
16     }
17
18     while ((c = fgetc(rfp)) != EOF) {
19         fputc(c, wfp);
20     }
21
22     fclose(rfp);
23     fclose(wfp);
24 }
```

실행

```
$ cat linux.txt
Linux System Programming
Hanbit Academy
$ ch4_11.out
$ cat linux.out
Linux System Programming
Hanbit Academy
```

- **08행, 13행** 08행에서 읽어올 파일을 열고, 13행에서 출력할 파일을 실행 파일 열기에 실패하면 NULL이 리턴
- **18~20행** fgetc() 함수를 이용해 EOF를 만날 때까지 한 문자씩 읽어들이 읽은 문자를 19행에서 fputc() 함수를 사용해 파일에 기록
- **22~23행** 모든 작업을 완료하면 열려 있는 두 파일을 파일 포인터를 사용해 닫음
- **실행 결과** linux.txt 파일과 linux.out 파일의 내용이 같음을 알 수 있음

03. 고수준 파일 입출력

■ 문자열 기반 입력 함수 : gets(3), fgets(3)

```
#include <stdio.h>
```

[함수 원형]

```
char *gets(const char *s);
```

```
char *fgets(char *s, int size, FILE *stream);
```

- s : 문자열을 저장한 버퍼의 시작 주소
- size : 버퍼의 크기
- stream : 파일 포인터
- gets() 함수의 특징
 - 표준 입력에서 문자열을 엔터키를 입력하거나 파일의 끝 을 만날 때까지 읽음
 - 읽어들이 문자열의 끝에서 개행 문자(엔터키 값)를 제외하고 널 문자('\0')를 채워 인자 s가 가리키는 영역에 저장하고 리턴
 - 메모리의 크기를 알 수 없기 때문에 s가 가득 찬 후에도 계속 읽을 수 있음
- fgets() 함수의 특징
 - 파일 포인터가 가리키는 파일에서 변수 size에 지정한 길이보다 하나 적게 문자열을 읽어 s에 저장
 - size의 길이만큼 읽는 도중에 개행 문자를 만나거나 파일의 끝을 만나면 해당 지점까지만 읽어옴
 - gets() 함수와 달리 개행 문자도 s에 저장 하고, 버퍼의 마지막 문자 다음에 널 문자를 저장

03. 고수준 파일 입출력

■ 문자열 기반 출력 함수 : puts(3), fputs(3)

```
#include <stdio.h>
```

[함수 원형]

```
int puts(const char *s);
```

```
int fputs(const char *s, FILE *stream);
```

- s : 문자열 주소
- stream : 파일 포인터
- puts() 함수의 특징
 - s가 가리키는 문자열을 표준 출력으로 출력, 이때 개행 문자를 추가해 출력
- fputs() 함수의 특징
 - s가 가리키는 문자열을 파일 포인터가 가리키는 파일로 출력
 - 출력할 때 개행 문자를 추가하지 않는다. 두 함수 모두 성공하면 음수가 아닌 수를 리턴하고, 파일의 끝이면 오류를 리턴

03. 고수준 파일 입출력

■ [예제 4-12] 문자열 기반 입출력 함수 사용하기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     FILE *rfp, *wfp;
06     int c;
07
08     if ((rfp = fopen("linux.txt", "r")) == NULL) {
09         perror("fopen: linux.txt");
10         exit(1);
11     }
12
13     if ((wfp = fopen("linux.out", "w")) == NULL) {
14         perror("fopen: linux.out");
15         exit(1);
16     }
17
18     while ((c = fgetc(rfp)) != EOF) {
19         fputc(c, wfp);
20     }
21
22     fclose(rfp);
23     fclose(wfp);
24 }
```

실행

```
$ ch4_12.out
$ cat linux.out
Linux System Programming
Hanbit Academy
Linux System Programming
Hanbit Academy
```

- **08행, 13행** 08행에서 읽어올 파일을 열고, 13행에서 출력할 파일을 실행 파일 열기에 실패하면 NULL이 리턴
- **18~20행** fgetc() 함수를 이용해 EOF를 만날 때까지 한 문자씩 읽어들이 읽은 문자를 19행에서 fputc() 함수를 사용해 파일에 기록
- **22~23행** 모든 작업을 완료하면 열려 있는 두 파일을 파일 포인터를 사용해 닫음
- **실행 결과** linux.txt 파일과 linux.out 파일의 내용이 같음을 알 수 있음

03. 고수준 파일 입출력

■ 버퍼 기반 입력 함수 : fread(3)

```
#include <stdio.h>
```

[함수 원형]

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

- ptr : 버퍼 주소
 - size : 버퍼 크기
 - nmemb : 읽어올 항목 수
 - stream : 파일 포인터
-
- fread() 함수의 특징
 - 항목의 크기가 size인 데이터를 nmemb에서 지정한 개수만큼 읽어 ptr이 가리키는 버퍼에 저장
 - 성공적으로 수행하면 읽어온 항목 수를 리턴
 - 읽을 항목이 없으면 0을, 파일의 끝을 만나면 EOF를 리턴

03. 고수준 파일 입출력

■ [예제 4-13] fread() 함수로 파일 읽기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     FILE *rfp;
06     char buf[BUFSIZ];
07     int n;
08
09     if ((rfp = fopen("linux.txt", "r")) == NULL) {
10         perror("fopen: linux.txt");
11         exit(1);
12     }
13
14     while ((n=fread(buf, sizeof(char)*2, 4, rfp)) > 0) {
15         buf[8] = '\0';
16         printf("n=%d, buf=%s\n", n, buf);
17     }
18
19     fclose(rfp);
20 }
```

실행

```
$ ch4_13.out
n=4, buf=Linux Sy
n=4, buf=stem Pro
n=4, buf=gramming
n=4, buf=
Hanbit
n=4, buf=Academy
```

- **14행** 항목의 크기를 sizeof(char)의 2배로 하고 한 번에 읽어올 개수를 4로 지정
char의 크기는 1이므로 실제로는 8(=2×4)바이트씩 읽어옴
- **15행** 문자열로 출력하기 위해 끝에 널 문자를 추가
- **실행 결과** 한 번에 8바이트씩 끊어서 읽어옴 리턴값을 저장한 n의 출력값을 보면
한 번에 4개 항목을 읽었음을 알 수 있음

03. 고수준 파일 입출력

■ 버퍼 기반 출력 함수 : fwrite(3)

```
#include <stdio.h>
```

[함수 원형]

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

- ptr : 버퍼 주소
 - size : 항목의 크기
 - nmemb : 항목 수
 - stream : 파일 포인터
-
- fwrite() 함수의 특징
 - 항목의 크기가 size인 데이터를 nmemb에서 지정한 개수만큼 ptr에서 읽어 stream으로 지정한 파일에 출력
 - 성공적으로 수행하면 출력한 항목의 수를, 오류가 발생 하면 EOF를 리턴

03. 고수준 파일 입출력

■ [예제 4-14] fwrite() 함수로 파일 출력하기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     FILE *rfp, *wfp;
06     char buf[BUFSIZ];
07     int n;
08
09     if ((rfp = fopen("linux.txt", "r")) == NULL) {
10         perror("fopen: linux.txt");
11         exit(1);
12     }
13
14     if ((wfp = fopen("linux.out", "a")) == NULL) {
15         perror("fopen: linux.out");
16         exit(1);
17     }
18
19     while ((n = fread(buf, sizeof(char)*2, 4, rfp)) > 0) {
20         fwrite(buf, sizeof(char)*2, n, wfp);
21     }
22
23     fclose(rfp);
24     fclose(wfp);
25 }
```

실행

```
$ ch4_14.out
$ cat linux.out
Linux System Programming
Hanbit Academy
Linux System Programming
Hanbit Academy
Linux System Programming
Hanbit Academy
```

- **14행** 출력할 파일을 추가로 실행
실행하면 [예제 4-12]의 실행 결과 파일(linux.out)에 덧붙여 출력
- **19행** 항목의 크기를 sizeof(char)의 2배로 하고 한 번에 읽어올 개수를 4로 지정
char의 크기는 1이므로 실제로는 2×4=8바이트씩 읽음
- **20행** 항목의 크기가 sizeof(char)×2인 항목을 19행에서 fread() 함수가 리턴한
항목 수인 만큼씩 wfp가 가리키는 파일에 출력
fwrite() 함수는 출력할 때 별도로 개행 문자를 추가 하지 않음
- **실행 결과** [예제 4-12]의 실행 결과 파일(linux.out)에 덧붙여 출력됨

03. 고수준 파일 입출력

■ 형식 기반 입출력

- 형식 기반 입출력 함수의 필요성
 - 만약 fgets()와 같은 함수를 사용해 파일을 읽어온다고 가정하면 과목별 점수를 구분해 떼어내는 작업을 해야 함
 - 고수준 파일 입출력은 이처럼 정해진 형식이 있는 파일을 읽을 때 유용하게 사용할 수 있는 형식 기반 입출력 함수를 제공

학번	영어	리눅스	게임	C 언어
2021001	80	95	80	95
2021002	85	90	90	80

03. 고수준 파일 입출력

■ 형식 기반 입력 함수 : scanf(3), fscanf(3)

```
#include <stdio.h>
```

[함수 원형]

```
int scanf(const char *format, ...);
```

```
int fscanf(FILE *stream, const char *format, ...);
```

- format : 입력 형식(%d, %s 등)
- stream : 파일 포인터
- scanf(), fscanf() 함수의 특징
 - scanf() 함수는 표준 입력에서 입력을 받고, fscanf() 함수는 지정한 파일에서 입력을 받음
 - 성공적으로 수행하면 읽은 항목의 개수를 리턴
 - 입력값이 형식에 맞지 않거나 너무 빨리 파일의 끝에 도달하면 0을 리턴
 - 형식에 맞는지 확인하기 전에 파일의 끝을 만나면 EOF를 리턴

03. 고수준 파일 입출력

■ [예제 4-15] fscanf() 함수로 파일 내용 읽기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     FILE *rfp;
06     int id, s1, s2, s3, s4, n;
07
08     if ((rfp = fopen("linux.dat", "r")) == NULL) {
09         perror("fopen: linux.dat");
10         exit(1);
11     }
12
13     printf("학번 평균\n");
14     while ((n=fscanf(rfp, "%d %d %d %d %d", &id,&s1,&s2,&s3,&s4)) != EOF) {
15         printf("%d : %d\n", id, (s1+s2+s3+s4)/4);
16     }
17     fclose(rfp);
18 }
```

실행

```
$ cat linux.dat
2021001 80 95 80 95
2021002 85 90 90 80
$ ch4_15.out
```

학번	평균
2021001	: 87
2021002	: 86

- **14행** 파일의 끝을 만날 때까지 입력 파일의 형식에 따라 학번과 성적을 한 행씩 읽음
읽어온 데이터는 이미 분리되어 있으므로 15행과 같이 별도의 작업 없이 바로 계산해 출력할 수 있음

03. 고수준 파일 입출력

■ 형식 기반 출력 함수 : printf(3), fprintf(3)

```
#include <stdio.h>
```

[함수 원형]

```
int printf(const char *format, ...);
```

```
int fprintf(FILE *stream, const char *format, ...);
```

- stream : 파일 포인터
- format : 출력 형식
- printf(), fprintf() 함수의 특징
 - printf() 함수는 표 준 출력으로 출력하고, fprintf() 함수는 지정한 파일로 출력
 - 출력 대상이 파일이기 때문에 첫 번째 인자가 파일 포인터가 된다는 점이 차이점
 - fprintf() 함수도 printf() 함수가 사용하는 형식 지정 방법을 그대로 사용
 - 두 함수 모두 성공적으로 수행하면 출력한 문자 수를, 오류가 발생하면 EOF를 리턴

03. 고수준 파일 입출력

■ [예제 4-16] fprintf() 함수를 사용해 지정한 파일로 출력하기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     FILE *rfp, *wfp;
06     int id, s1, s2, s3, s4, n;
07
08     if ((rfp = fopen("linux.dat", "r")) == NULL) {
09         perror("fopen: linux.dat");
10         exit(1);
11     }
12
13     if ((wfp = fopen("linux.scr", "w")) == NULL) {
14         perror("fopen: linux.scr");
15         exit(1);
16     }
17
18     fprintf(wfp, "학번 평균\n");
19     while ((n=fscanf(rfp, "%d %d %d %d %d", &id,&s1,&s2,&s3,&s4)) != EOF) {
20         fprintf(wfp, "%d : %d\n", id, (s1+s2+s3+s4)/4);
21     }
22
23     fclose(rfp);
24     fclose(wfp);
25 }
```

실행

```
$ ch4_16.out
$ cat linux.scr
학번      평균
2021001 : 87
2021002 : 86
```

- **13행** linux.scr 파일을 쓰기 모드로 생성
- **18행** 항목의 제목을 파일에 출력
- **19행** 파일의 끝까지 입력 파일의 형식에 따라 학번과 성적을 한 행씩 읽음
읽어온 데이터는 이미 분리되어 있으므로 20행과 같이 별도의 작업 없이 바로 계산해 파일 포인터 wfp가 가리키는 파일에 출력
- **실행 결과** linux.scr 파일에 학번과 평균이 저장

03. 고수준 파일 입출력

■ 파일 오프셋 이동 : fseek(3)

```
#include <stdio.h>
```

[함수 원형]

```
int fseek(FILE *stream, long offset, int whence);
```

- stream : 파일 포인터
- offset : 이동할 오프셋
- whence : 오프셋의 기준 위치
- fseek() 함수의 특징
 - lseek() 함수와 유사하게 동작
 - fseek() 함수는 stream이 가리키는 파일에서 offset에 지정한 크기만큼 오프셋을 이동
 - whence는 offset 값을 해석하는 방법을 결정하는 상수로, lseek() 함수에서와 같은 의미
 - whence에 사용할 수 있는 값
 - SEEK_SET : 파일의 시작을 기준으로 계산
 - SEEK_CUR : 현재 위치를 기준으로 계산
 - SEEK_END : 파일의 끝을 기준으로 계산
 - 성공하면 변경된 오프셋을 리턴하지만 fseek() 함수는 성공하면 0을, 실패하면 EOF를 리턴
 - 잘못된 예시

```
cur = fseek(fp, 0, SEEK_CUR);      (×)
```

03. 고수준 파일 입출력

■ 현재 오프셋 구하기 : ftell(3)

```
#include <stdio.h>
```

[함수 원형]

```
long ftell(FILE *stream);
```

- stream : 파일 포인터
- ftell() 함수의 특징
 - 인자로 지정한 파일의 현재 오프셋을 리턴
 - ftell() 함수가 리턴하는 오프셋은 파일의 시작에서 현재 오프셋 위치까지의 바이트 수
 - 수행에 실패하면 EOF를 리턴, 따라서 현재 오프셋 값을 구하려면 ftell() 함수를 다음과 같이 사용

```
long cur;  
cur = ftell(fp);
```

03. 고수준 파일 입출력

■ 처음 위치로 오프셋 이동 : rewind(3)

```
#include <stdio.h>
```

[함수 원형]

```
void rewind(FILE *stream);
```

- stream : 파일 포인터
- rewind() 함수의 특징
 - 오프셋 위치를 파일의 시작으로 즉시 이동

03. 고수준 파일 입출력

■ [예제 4-17] 오프셋 이동하며 읽어서 출력하기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     FILE *fp;
06     int n;
07     long cur;
08     char buf[BUFSIZ];
09
10     if ((fp = fopen("linux.txt", "r")) == NULL) {
11         perror("fopen: linux.txt");
12         exit(1);
13     }
14
15     cur = ftell(fp);
16     printf("Offset cur=%d\n", (int)cur);
17
18     n = fread(buf, sizeof(char), 5, fp);
19     buf[n] = '\0';
20     printf("-- Read Str=%s\n", buf);
21
22     fseek(fp, 1, SEEK_CUR);
23
24     cur = ftell(fp);
25     printf("Offset cur=%d\n", (int)cur);
26
27     n = fread(buf, sizeof(char), 6, fp);
28     buf[n] = '\0';
29     printf("-- Read Str=%s\n", buf);
30
31     fseek(fp, 1, SEEK_CUR);
32
33     cur = ftell(fp);
34     printf("Offset cur=%d\n", (int)cur);
35
36     n = fread(buf, sizeof(char), 11, fp);
37     buf[n] = '\0';
38     printf("-- Read Str=%s\n", buf);
39
40     rewind(fp);
41     cur = ftell(fp);
42     printf("Rewind Offset cur=%d\n", (int)cur);
43
44     fclose(fp);
45 }
```

실행

```
$ ch4_17.out
Offset cur=0
-- Read Str=Linux
Offset cur=6
-- Read Str=System
Offset cur=13
-- Read Str=Programming
Rewind Offset cur=0
```

- **15행, 24행, 33행, 41행** ftell() 함수로 현재 오프셋을 확인
- **16행, 25행, 34행, 42행** ftell() 함수로 확인한 오프셋을 출력
- **22행, 31행** 현재 오프셋을 기준으로 1만큼 이동한다. 18행에서 5바이트를 읽어 현재 오프셋이 5였는데, 22행을 실행한 결과 오프셋이 6으로 이동
이 오프셋은 "Linux SystemProgramming"에서 'S'에 해당하는 위치로 31행의 경우 'P'로 이동
- **40행** 첫 행을 출력한 후 rewind() 함수를 사용해 오프셋을 다시 시작 위치로 이동
- **실행 결과** 오프셋이 이동하면서 단어별로 출력되는 것을 확인할 수 있음

03. 고수준 파일 입출력

■ 파일과 디스크 동기화 함수: fflush(3)

```
#include <stdio.h>
```

[함수 원형]

```
int fflush(FILE *stream);
```

- stream : 파일 포인터
- fflush(3) 함수의 특징
 - 버퍼에 있는 데이터를 파일에 기록
 - 파일을 읽기 전용으로 연 경우 버퍼에 있는 내용을 모두 비움
 - 파일 포인터가 NULL이면 쓰기 전용으로 연 모든 파일에 데이터를 사용

04. 파일 기술자와 파일 포인터 변환

■ 파일 기술자와 파일 포인터 변환

- 저수준 파일 입출력에서는 열린 파일을 가리킬 때 파일 기술자를 사용
- 고수준 파일 입출력에서는 파일 포인터를 사용
- 파일 기술자에서 파일 포인터를 생성하려면 `fdopen()` 함수를 사용
- 파일 포인터에서 파일 기술자 정보를 추출하려면 `fileno()` 함수를 사용

표 4-6 파일 기술자와 파일 포인터 변환

기능	함수
파일 기술자 → 파일 포인터	<code>FILE *fdopen(int fd, const char *mode);</code>
파일 포인터 → 파일 기술자	<code>int fileno(FILE *stream);</code>

04. 파일 기술자와 파일 포인터 변환

■ 파일 포인터 생성 : fdopen(3)

```
#include <stdio.h>
```

[함수 원형]

```
FILE *fdopen(int fd, const char *mode);
```

- fd : 파일 기술자
- mode : 열기 모드
- fdopen() 함수의 특징
 - 파일 기술자와 모드(읽기, 쓰기 등)를 인자로 받아 파일 포인터를 생성
 - mode에 사용할 수 있는 값은 fopen()에서 사용한 값([표 4-5] 참고)
 - mode에 값을 지정할 때는 파일 기술자를 열 때와 같은 종류의 값으로 지정해야 함
 - 저수준에서는 읽기 전용(O_RDONLY)으로 열었는데, 고수준으로 변환하면서 쓰기(w)로 지정하면 정상적으로 동작하지 않음
 - fdopen() 함수는 성공하면 파일 포인터를, 실패하면 널 포인터를 리턴

04. 파일 기술자와 파일 포인터 변환

■ [예제 4-18] fdopen() 함수로 파일 포인터 생성하기

```
01 #include <fcntl.h>
02 #include <stdlib.h>
03 #include <stdio.h>
04
05 int main() {
06     FILE *fp;
07     int fd;
08     char str[BUFSIZ];
09
10     fd = open("linux.txt", O_RDONLY);
11     if (fd == -1) {
12         perror("open");
13         exit(1);
14     }
15
16     fp = fdopen(fd, "r");
17
18     fgets(str, BUFSIZ, fp);
19     printf("Read : %s\n", str);
20
21     fclose(fp);
22 }
```

실행

\$ ch4_18.out

Read : Linux System Programming

- **10행** 저수준 파일 입출력 함수인 `open()`으로 파일을 실행
- **16행** `fdopen()` 함수를 사용해 파일 포인터를 생성
- **18행** 16행에서 생성한 파일 포인터를 통해 고수준 파일 입출력 함수로 데이터를 읽음
- **실행 결과** `fgets()` 함수에 파일 포인터가 정상적으로 전달되어 데이터를 읽고 출력했음을 알 수 있음

04. 파일 기술자와 파일 포인터 변환

■ 파일 포인터 생성 : `fopen()`

```
#include <stdio.h>
```

[함수 원형]

```
int fopen(FILE *stream);
```

- `stream` : 파일 포인터
- `fopen()` 함수의 특징
 - 파일 포인터를 인자로 받아 파일 기술자를 리턴

04. 파일 기술자와 파일 포인터 변환

■ [예제 4-19] fileno() 함수로 파일 기술자 추출하기

```
01 #include <unistd.h>
02 #include <fcntl.h>
03 #include <stdlib.h>
04 #include <stdio.h>
05
06 int main() {
07     FILE *fp;
08     int fd, n;
09     char str[BUFSIZ];
10
11     fp = fopen("linux.txt", "r");
12     if (fp == NULL) {
13         perror("fopen");
14         exit(1);
15     }
16
17     fd = fileno(fp);
18     printf("fd : %d\n", fd);
19
20     n = read(fd, str, BUFSIZ);
21     str[n] = '\0';
22     printf("Read : %s\n", str);
23
24     close(fd);
25 }
```

실행

```
$ ch4_19.out
fd : 3
Read : Linux System Programming
Hanbit Academy
```

- **17행** 11행에서 fopen() 함수로 파일을 열고 리턴받은 파일 포인터에서 파일 기술자를 추출
실행 결과에서 볼 수 있듯이 파일 기술자는 3
- **20행** 파일 기술자를 이용해 저수준 파일 입출력 함수로 데이터를 읽음
- **21~22행** 20행에서 읽은 데이터에는 문자열의 끝을 표시하는 널이 포함되지 않으므로
강제로 널 값을 삽입하고 출력
- **실행 결과** 파일 기술자가 3번으로 할당되고, linux.txt 파일의 내용을 모두 읽고 출력했음을
알 수 있음

05. 임시 파일 사용

■ 임시 파일명 생성 : tmpnam(3)

```
#include <stdio.h>
```

[함수 원형]

```
char *tmpnam(char *s);
```

- s : 파일명을 저장할 버퍼의 시작 주소
- tmpnam() 함수의 특징
 - 임시 파일명을 시스템이 알아서 지정
 - 인자가 있으면 경우 해당 인자가 가리키는 곳에 임시 파일명을 저장하고, 인자가 NULL이면 임시 파일명을 리턴

05. 임시 파일 사용

■ 템플릿을 지정한 임시 파일명 생성 : mktemp(3)

```
#include <stdlib.h>
```

[함수 원형]

```
char *mktemp(char *template);
```

- template : 임시 파일명의 템플릿
- mktemp() 함수의 특징
 - 임시 파일의 템플릿을 인자로 받아 이를 임시 파일명으로 변환해 리턴
 - 인자로 지정하는 템플릿은 대문자 X 6개로 마쳐야 함
 - 예시

```
/tmp/hanbitXXXXXX
```

- mktemp() 함수는 대문자 X 부분을 다른 문자로 대체해 임시 파일명을 만듦
- 임시 파일명 생성에 실패하면 널 문자열을 리턴

05. 임시 파일 사용

■ [예제 4-20] 임시 파일명 만들기

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04
05 int main() {
06     char *fname;
07     char fntmp[BUFSIZ];
08     char template[32];
09
10     fname = tmpnam(NULL);
11     printf("1. TMP File Name(tmpnam) : %s\n", fname);
12
13     tmpnam(fntmp);
14     printf("2. TMP File Name(tmpnam) : %s\n", fntmp);
15
16     strcpy(template, "/tmp/hanbitXXXXXX");
17     fname = mktemp(template);
18     printf("3. TMP File Name(mktemp) : %s\n", fname);
19 }
```

실행

\$ ch4_20.out

1. TMP File Name(tmpnam) : /tmp/fileZslWTk
2. TMP File Name(tmpnam) : /tmp/file04AV0o
3. TMP File Name(mktemp) : /tmp/hanbitVtWCZk

- **10행** tmpnam() 함수에 인자를 NULL로 지정해 임시 파일명을 리턴값으로 받음
- **13행** 인자로 지정한 배열에 임시 파일명을 저장
- **16~20행** 임시 파일명의 템플릿을 생성하고 mktemp()로 임시 파일명을 생성
- **실행 결과** 임시 파일은 모두 /tmp 디렉터리 아래에 생성됨

[예제 4-20]을 컴파일해보면 경고 메시지가 나올 수 있는데, 이는 이 함수들이 보안이 취약할 수 있기 때문

05. 임시 파일 사용

■ 임시 파일의 파일 포인터와 파일 기술자 생성: tmpfile(), mkstemp()

```
#include
```

[함수 원형]

```
FILE *tmpfile();
```

```
#include <stdlib.h>
```

[함수 원형]

```
int mkstemp(char *template);
```

- template : 임시 파일명의 템플릿
- tmpfile(), mkstemp() 함수의 특징
 - 대부분의 경우 프로그램을 수행할 때 임시 파일명을 반드시 알아야 하는 것은 아님
 - 임시 파일에 대한 파일 포인터만 필요하면 tmpfile() 함수를 사용하고, 파일 기술자가 필요하면 mkstemp() 함수를 사용
 - tmpfile() 함수는 자동으로 w+ 모드로 열린 파일 포인터를 리턴
 - mkstemp() 함수는 템플릿으로 임시 파일명을 지정해 열고 파일 기술자를 리턴
 - 이 함수들로 생성한 파일 포인터와 파일 기술자는 바로 입출력에 사용할 수 있음

05. 임시 파일 사용

■ [예제 4-21] tmpfile(), mkstemp() 함수로 임시 파일 생성하기

```
01 #include <string.h>
02 #include <unistd.h>
03 #include <stdlib.h>
04 #include <stdio.h>
05
06 int main() {
07     FILE *fp;
08     int fd;
09     char template[32];
10     int ret;
11
12     fp = tmpfile();
13     fputs("Temp File", fp);
14     fclose(fp);
15
16     strcpy(template, "/tmp/hanbitXXXXXX");
17     fd = mkstemp(template);
18     write(fd, "Temp File", 10);
19     close(fd);
20 }
```

실행

\$ ch4_20.out

1. TMP File Name(tmpnam) : /tmp/fileZslWTk
2. TMP File Name(tmpnam) : /tmp/file04AV0o
3. TMP File Name(mktemp) : /tmp/hanbitVtWCZk

- **12행** 임시 파일을 생성하고 파일 포인터를 리턴
- **13행** 생성한 임시 파일에 fputs() 함수를 사용해 "Temp File"이라는 문자열을 출력
- **16행** 임시 파일명의 템플릿을 지정
- **17행** 템플릿을 사용해 임시 파일을 생성하고 파일 기술자를 리턴
- **18행** write() 함수를 사용해 임시 파일에 "Temp File"이라는 문자열을 출력 .
- **실행 결과** 프로그램 내부에서 임시로 파일을 생성했다가 종료하는 것이므로 결과로 확인할 수 없음
프로그램 내에서 문자 그대로 임시로 사용하는 것

시스템 프로그래밍

리눅스&유닉스

감사합니다.
